

Helix AI: Backend Code Walkthrough (In-Depth)

From API endpoint to validated modernization output

Generated: 2026-04-05 01:24:58

This report teaches the backend from scratch in simple language, then dives into module-level implementation details.

Project Snapshot	Value
Backend files	14
Backend LOC	1158
API endpoints	10
Backend LOC	1158
Frontend LOC	2647
Test LOC	157
Detected endpoints	10
Migration transform rules	15
Rule benchmark pass	4/4
Unit test status	pass (10 tests)

How to read this document: each section starts with a simple explanation, then a deeper engineering explanation, then a practical checklist.

0. Backend Structure Overview

Simple Explanation

In simple words: main.py wires all agents and cores, while each module handles one concern.

Deep Explanation

Backend currently spans 14 core Python files with about 1158 non-empty lines.

Code analysis finds roughly 58 functions and 17 classes in backend + tests.

This modular shape makes it straightforward to explain and extend in academic defense.

Practical Checklist

- Keep module boundaries clear.
- Avoid business logic in route handlers where possible.
- Preserve pure helper functions for testability.

Code References

```
main.py
agents/
core/
```

1. API Design

Simple Explanation

In simple words: the API has dedicated routes for health, model status, analysis, refactor, and run history.

Deep Explanation

GET /health and GET /model-status provide operations visibility.

POST /analyze provides non-destructive planning output.

POST /refactor executes guarded modernization and returns diff/report.

GET /runs and GET /runs/{id} support run timeline UX.

Practical Checklist

- Keep analyze and refactor separate for user trust.
- Return consistent JSON structures.
- Use proper HTTP errors for missing runs or failures.

Code References

```
main.py
```

2. Analyzer Internals

Simple Explanation

In simple words: analyzer uses explicit regex rules and small AST checks to detect legacy patterns quickly.

Deep Explanation

Current analyzer holds around 15 legacy-pattern rules and risk mode escalation logic.

Source version inference is evidence-based: issue hints determine whether input looks like Python 1/2 or Python 3.

The analyzer also emits semantic risk signals for division and text/bytes boundaries.

Practical Checklist

- Keep every rule documented with id and replacement hint.
- Prefer transparent rules over hidden heuristics for first release.
- Add new rules with paired tests.

Code References

`agents/analyzer.py`

3. Suggestion, Critique, Planning

Simple Explanation

In simple words: suggestion maps issue to action, critic scores risk, planner decides execution order.

Deep Explanation

Suggestion mapping uses a deterministic issue-to-suggestion table with confidence and priority values.

Critic applies type-based base score and penalties for semantic or low-confidence operations.

Planner computes final score = `safety_score + max(0, 50 - priority)`, then sorts by order and score.

Practical Checklist

- Keep scoring function simple and explainable.
- Use warning status to surface review needs.
- Preserve deterministic sort order for repeatability.

Formulas

```
final_score = critique_safety_score + max(0, 50 - suggestion_priority)
selected_plan = candidate where suggestion_type != 'NOOP'
```

Code References

```
agents/suggester.py
agents/critic.py
agents/planner.py
```

4. Execution Core

Simple Explanation

In simple words: execution applies only known transforms, one by one, using explicit rule ids.

Deep Explanation

The execution core currently includes 15 explicit transformation branches.

Most transforms are regex-based; some (`print/exec/apply/backticks`) use dedicated helper methods.

Sequential application allows ordered rewrites and predictable diff output.

Practical Checklist

- Never run unknown transform ids.
- Prefer idempotent rewrite functions when possible.
- Keep special cases as helper methods, not giant regex.

Code References

```
core/execution.py
```

5. Validation Core

Simple Explanation

In simple words: validation is the final gate that decides pass or rollback.

Deep Explanation

Validation stages: syntax parse, leftover-legacy lint checks, and behavior checks.

Syntax parse prevents invalid output from being accepted.

Legacy remainder patterns prevent partial migration from silently passing.

Behavior check compares function-name sets between original and migrated code.

Practical Checklist

- Block output if syntax fails.
- Block output if leftover legacy constructs remain.
- Return warnings for manual semantic review areas.

Code References

`core/validation.py`

6. Report + Run Persistence

Simple Explanation

In simple words: every run can be inspected later with report, diff, and metadata.

Deep Explanation

Report generator produces a markdown report with analysis, planned transformations, semantic risks, and validation summary.

Run store writes JSONL records for append-only history and quick retrieval.

This supports evidence-based demos and debugging sessions.

Practical Checklist

- Keep report content human-readable.
- Avoid storing only aggregate numbers; keep context too.
- Use run ids as stable references.

Code References

```
core/report_generator.py
core/run_store.py
```

7. ML Runtime Integration

Simple Explanation

In simple words: ML path can be enabled with environment variables and adapter files, without breaking deterministic path.

Deep Explanation

Model status and availability are visible in API responses and health checks.

If adapter path is missing, system gracefully reports unavailable ML instead of crashing pipeline.

Inference prompt is explicit and constrained to code modernization task.

Practical Checklist

- Keep ML dependencies optional.
- Always expose model status for debugging.
- Treat ML output as auxiliary unless validated.

Code References

```
core/config.py
core/ml_reasoner.py
```

8. Backend Improvement Plan

Simple Explanation

In simple words: next backend upgrades are stronger AST rewrites, richer metrics, and CI checks.

Deep Explanation

Add AST-level transforms for complex patterns to reduce regex false positives.

Add per-stage latency metrics and warning counters.

Integrate pre-commit linting and CI test execution for faster regression detection.

Practical Checklist

- Prioritize correctness before adding transform count.
- Instrument each stage for observability.
- Keep backward-compatible API responses.

Code References

tests/
core/
agents/

Viva Quick Answers

Q: Why FastAPI?

It gives fast route development, typed request models, and straightforward JSON responses.

Q: Where are transformations implemented?

In core/execution.py as explicit operation handlers.

Q: How do you guarantee correctness?

Validation core checks syntax, leftover legacy patterns, and behavior proxies before acceptance.

Q: How is history stored?

Append-only JSONL via RunStore for simplicity and auditability.

Q: How can someone add a new rule?

Add analyzer pattern, suggester mapping, execution transform, and a dedicated test case.

From Scratch Learning Track

This chapter is written for beginners first. Each module starts with very simple meaning, then engineering depth, then practical action.

Module 1: Fastapi Structure And Route Boundaries

Simple view: FastAPI structure and route boundaries explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, FastAPI structure and route boundaries connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 2: Request And Response Model Consistency

Simple view: request and response model consistency explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, request and response model consistency connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 3: Analysis Endpoint Behavior

Simple view: analysis endpoint behavior explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, analysis endpoint behavior connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 4: Refactor Endpoint Behavior

Simple view: refactor endpoint behavior explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, refactor endpoint behavior connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 5: Execution Transform Implementation Style

Simple view: execution transform implementation style explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, execution transform implementation style connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 6: Validation Gate Design

Simple view: validation gate design explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, validation gate design connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 7: Report Generation Structure

Simple view: report generation structure explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, report generation structure connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 8: Run Persistence And Retrieval

Simple view: run persistence and retrieval explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, run persistence and retrieval connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 9: Config And Environment Management

Simple view: config and environment management explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, config and environment management connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 10: Error Handling Conventions

Simple view: error handling conventions explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, error handling conventions connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 11: Testability And Module Boundaries

Simple view: testability and module boundaries explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, testability and module boundaries connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 12: Backend Roadmap With Measurable Milestones

Simple view: backend roadmap with measurable milestones explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, backend roadmap with measurable milestones connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic

and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Hands-On Practical Labs

1. Lab 1: Baseline Input Audit

Start from a legacy snippet and identify exactly what will break on modern Python.

- Collect one legacy input sample.
- Run /analyze and record issue ids.
- Classify each issue as syntax or semantic risk.
- Write one paragraph on expected modernization output.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

2. Lab 2: Analyze Output Interpretation

Understand how analysis payload maps to later planner and execution actions.

- Inspect probable_source_version and mode.
- Read legacy_issues list line by line.
- Map each issue id to a suggestion id.
- Explain why some items should become warning.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

3. Lab 3: Critic + Planner Trace

Trace how safety score and priority become selected_plans.

- Run analyze output through critique logic.
- Compute final score manually for two suggestions.
- Verify execution order in planner output.
- Compare manual and program outputs.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

4. Lab 4: Execution and Diff Review

Run modernization and inspect unified diff before applying changes.

- Execute /refactor on sample input.
- Inspect added and removed lines.
- Confirm each line links to a planned transform.
- Document one risky change for manual review.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

5. Lab 5: Validation Failure Drill

Learn fail-closed behavior by forcing a leftover-legacy construct.

- Create code where one legacy construct remains.
- Run validation and capture failure stage.
- Explain rollback behavior and user impact.
- Fix rule and rerun to green state.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

6. Lab 6: Run History and Reproducibility

Use run history to prove the pipeline is auditable and repeatable.

- Generate at least two runs with different inputs.
- Use /runs and /runs/{id} endpoints.
- Re-open one run in UI and inspect report tab.
- Write traceability summary notes.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

7. Lab 7: Topic-Specific Deep Task

Perform a deeper exercise related to backend concerns and document engineering tradeoffs.

- Choose one module strongly tied to this topic.
- Read code and summarize control flow.
- List strengths, risks, and missing checks.
- Propose one measurable improvement.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

8. Lab 8: Evaluation and Evidence Pack

Prepare evidence artifacts you can use in submission and viva.

- Run tests and benchmark script.
- Capture summary metrics and limitations.
- Align report claims with actual metrics.
- Store outputs in a clean docs folder.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

9. Lab 9: Explain to Beginner

Re-explain one complex part in simple words without losing correctness.

- Pick one algorithmic decision.
- Write a beginner explanation in 8 lines.
- Write an expert explanation in 8 lines.
- Ensure both versions match same truth.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

10. Lab 10: Full Demo Rehearsal

Do complete end-to-end dry run as if in final presentation.

- Start server and open UI.
- Run one safe case and one warning case.
- Show diff, report, and run history.
- Conclude with roadmap and limitations.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

Troubleshooting Playbook

Issue	Likely Cause	How To Fix
Analysis returns no issues unexpectedly	Input pattern not covered by rules	Add/adjust analyzer rule and unit test.
Planner selects zero actions	Suggestions became NOOP or filtered by status	Inspect suggestion ids, critique map, and planner filters.
Validation fails after execution	Legacy leftovers or syntax problems remain	Inspect failure stage and patch transform logic.
Diff looks confusing to users	Output formatting or context lines are unclear	Improve diff rendering and report hints.
Run history missing expected data	RunStore payload fields not persisted	Verify run payload contract in save_run call.
ML shows unavailable	Adapter path missing or model disabled	Check env flags and adapter directory path.
UI button state feels wrong	State machine and action label out of sync	Trace state.primaryAction transitions.
Performance slows on large input	Heavy rendering or repeated processing	Paginate rendering and profile hot paths.
Benchmarks look too optimistic	Benchmark set too small or narrow	Expand dataset and stratify by pattern type.
Viva answers feel weak	No concrete evidence references	Quote tests, benchmark numbers, and code paths.
Topic-specific confusion in endpoint flow	Conceptual information in headers	Add diagram plus file-level walkthrough.
Claims sound exaggerated	Scope and limitations not stated	Add explicit limitations and roadmap stages.

Glossary (Simple Words, Technical Meaning)

1. Legacy Pattern

An old syntax or API usage from earlier Python versions.

2. Modernization

Converting legacy code to current Python-compatible code.

3. Deterministic

Same input always gives same output.

4. Semantic Risk

Code may still run but behavior could change.

5. Validation Gate

Final checks that decide pass or rollback.

6. Diff-First Review

Inspect exact changes before applying to editor.

7. Analyzer

Stage that detects issues and risk indicators.

8. Suggester

Stage that maps detected issues to upgrades.

9. Critic

Stage that assigns safety scores and warnings.

10. Planner

Stage that chooses execution order.

11. Execution Core

Stage that applies selected transforms.

12. RunStore

Persistence layer for run history records.

13. LOCA (LOC)

Lines of code metric for rough project size.

14. Benchmark

Set of test cases used to measure pass rate.

15. Smoke Test

Small test set for quick health verification.

16. Rollback

Revert to original code when validation fails.

17. Adapter

LoRA fine-tuned parameter delta for base model.

18. Base Model

Original pretrained model used before fine-tuning.

19. Confidence

Estimated reliability signal for a suggestion.

20. Traceability

Ability to explain what changed and why.

21. Endpoint Contract

Expected request and response fields for API.

22. Idempotent

Operation that can repeat without new side effects.

23. Schema Stability

Keeping response shape consistent across updates.

24. Guard Clause

Early return for invalid or unsafe state.

25. Service Layer

Modules that hold business logic outside routes.

Extended Viva and Interview Q&A

Q1: What is the core goal of this report?

To explain this topic from beginner level to implementation depth with real project evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q2: How should I present this in viva?

Start with simple explanation, then show one concrete code path, then show one metric and one limitation.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q3: How do I avoid vague answers?

Always tie statements to file paths, endpoints, tests, or measured outputs.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q4: What if evaluator asks about limitations?

Answer directly and show roadmap with near-term measurable upgrades.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q5: How do I show project credibility?

Use deterministic workflow, validation behavior, and test/benchmark evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q6: Why use simple language in technical docs?

It improves clarity and comprehension without reducing technical correctness.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q7: How should I practice this topic?

Run one lab task, inspect code, and explain output in your own words.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q8: How does this topic connect to full system?

Each topic is one layer of the same end-to-end modernization pipeline.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q9: What is the most critical backend path?

The /refactor route because it executes full guarded pipeline and returns final artifacts.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q10: How do you keep backend maintainable?

By isolating concerns into modules and keeping response contracts stable.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q11: What is first backend improvement you would do?

Add more AST-based transforms where regex is fragile.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Code Walkthrough Appendix (With Explanation)

This appendix connects explanation to real files. Each excerpt includes simple-language meaning, technical interpretation, and why it matters before showing code lines.

Excerpt 1: API + Pipeline

File: /Users/rutwikbhondave/Desktop/Helix AI Project/main.py

Simple explanation: this block handles API orchestration and full modernization flow control. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=14, classes=2, endpoint decorators=8.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: read_landing, health_check, model_status, list_runs, read_landing, health_check, model_status, list_runs.
- Endpoint decorators in excerpt: app.get, app.get, app.get, app.get, app.get.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import difflib
0002: import json
0003: import logging
0004: import os
0005: from datetime import datetime
0006:
0007: import uvicorn
0008: from fastapi import FastAPI, HTTPException, Request
0009: from fastapi.responses import HTMLResponse, JSONResponse
0010: from fastapi.staticfiles import StaticFiles
0011: from fastapi.templating import Jinja2Templates
0012: from pydantic import BaseModel
0013:
0014: from agents.analyzer import AnalyzerAgent
0015: from agents.critic import CriticAgent
0016: from agents.planner import PlannerAgent
0017: from agents.suggester import SuggestionAgent
0018: from core.config import settings
0019: from core.execution import ExecutionCore
0020: from core.ml_reasoner import MLReasoner
0021: from core.report_generator import ReportGenerator
0022: from core.run_store import RunStore
0023: from core.validation import ValidationCore
0024:
0025:
0026: analyzer = AnalyzerAgent()
0027: suggester = SuggestionAgent()
0028: critic = CriticAgent()
0029: planner = PlannerAgent()
0030: executor = ExecutionCore()
0031: validator = ValidationCore()
```

```

0032: reporter = ReportGenerator()
0033: run_store = RunStore(settings.RUN_HISTORY_FILE)
0034: reasoner = MLReasoner(
0035:     enabled=settings.ML_MODEL_ENABLED,
0036:     adapter_path=os.path.join(settings.BASE_DIR, settings.ML_MODEL_ADAPTER_PATH)
0037:     if not os.path.isabs(settings.ML_MODEL_ADAPTER_PATH)
0038:     else settings.ML_MODEL_ADAPTER_PATH,
0039:     base_model=settings.ML_MODEL_BASE,
0040: )
0041:
0042: app = FastAPI(
0043:     title=settings.APP_NAME,
0044:     version=settings.VERSION,
0045:     docs_url=None,
0046:     redoc_url=None,
0047: )
0048:
0049: app.mount("/static", StaticFiles(directory=settings.STATIC_DIR), name="static")
0050: templates = Jinja2Templates(directory=settings.TEMPLATE_DIR)
0051:
0052:
0053: class CodeRequest(BaseModel):
0054:     code: str
0055:
0056:
0057: class ContactRequest(BaseModel):
0058:     name: str
0059:     email: str
0060:     project: str
0061:     message: str
0062:
0063:
0064: INFO_PAGES = {
0065:     "mission": {
0066:         "title": "Our Mission",
0067:         "content": "<p>Helix AI modernizes legacy Python code with an autonomous multi-agent work
0068:     },
0069:     "team": {
0070:         "title": "The Team",
0071:         "content": "<p>A platform prototype built around agentic planning, rule-based modernization
0072:     },
0073:     "newsletter": {
0074:         "title": "Helix AI Updates",
0075:         "content": "<p>Track new migration rules, supported legacy patterns, and autonomous valida
0076:     },
0077:     "careers": {
0078:         "title": "Join Us",
0079:         "content": "<p>We are focused on developer tooling, code migration, and safe autonomous ex
0080:     },
0081:     "contact": {
0082:         "title": "Contact Us",
0083:         "content": "<p>Questions about legacy Python modernization, agentic workflows, or enterpr
0084:     },
0085:     "refund-policy": {
0086:         "title": "Project Scope",
0087:         "content": "<p>Helix AI focuses on legacy Python modernization. The current prototype str
0088:     },
0089:     "faqs": {
0090:         "title": "Frequently Asked Questions",
0091:         "content": "<p><strong>Q: What does Helix AI do today?</strong><br>A: It analyzes legacy P
0092:     },
0093:     "stories": {
0094:         "title": "Use Cases",
0095:         "content": "<p>Helix AI is designed for teams modernizing older Python services, education
0096:     },
0097:     "status": {
0098:         "title": "System Status",
0099:         "content": "<p>Modernization pipeline online.</p><ul class='list-disc pl-5 mt-4 space-y-2
0100:     },

```

```

0101: }
0102:
0103:
0104: @app.get("/", response_class=HTMLResponse)
0105: async def read_landing(request: Request):
0106:     return templates.TemplateResponse("new.html", {"request": request})
0107:
0108:
0109: @app.get("/health")
0110: async def health_check():
0111:     return JSONResponse(
0112:         content={
0113:             "status": "ok",
0114:             "app": settings.APP_NAME,
0115:             "version": settings.VERSION,
0116:             "ml_reasoner": reasoner.explain_status(),
0117:         }
0118:     )
0119:
0120:
0121: @app.get("/model-status")
0122: async def model_status():
0123:     return JSONResponse(content=reasoner.explain_status())
0124:
0125:
0126: @app.get("/runs")
0127: async def list_runs(limit: int = 10):
0128:     return JSONResponse(content={"runs": run_store.list_runs(limit=max(1, min(limit, 50)))})
0129:
0130:
0131: @app.get("/runs/{run_id}")
0132: async def get_run(run_id: str):
0133:     record = run_store.get_run(run_id)
0134:     if not record:
0135:         raise HTTPException(status_code=404, detail="Run not found")
0136:     return JSONResponse(content=record)
0137:
0138:
0139: @app.get("/app", response_class=HTMLResponse)
0140: async def read_app(request: Request):
0141:     return templates.TemplateResponse("index.html", {"request": request, "app_name": settings.APP_NAME})
0142:
0143:
0144: @app.get("/info/{page_name}", response_class=HTMLResponse)
0145: async def read_info_page(request: Request, page_name: str):
0146:     page_data = INFO_PAGES.get(page_name)
0147:     if not page_data:
0148:         raise HTTPException(status_code=404, detail="Page not found")
0149:
0150:     return templates.TemplateResponse(
0151:         "info.html",
0152:         {
0153:             "request": request,
0154:             "page_title": page_data["title"],
0155:             "content": page_data["content"],
0156:         },
0157:     )
0158:
0159:
0160: @app.post("/submit-request")

```

Excerpt 2: Execution Core

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/execution.py

Simple explanation: this block handles deterministic modernization rewrite engine. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=7, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `apply_changes`, `_apply_single`, `_upgrade_print_statements`, `_upgrade_backtick_repr`.
- Regex operations are present, so edge-case pattern handling should be tested carefully.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import re
0002: from typing import List
0003:
0004:
0005: class ExecutionCore:
0006:     def apply_changes(self, code: str, plans) -> str:
0007:         if not plans:
0008:             return code
0009:
0010:         if isinstance(plans, dict):
0011:             plans = [plans]
0012:
0013:         candidate = code
0014:         for plan in plans:
0015:             candidate = self._apply_single(candidate, plan["suggestion"] if "suggestion" in plan else plan["operation"])
0016:         return candidate
0017:
0018:     def _apply_single(self, code: str, suggestion: dict) -> str:
0019:         op = suggestion["id"]
0020:
0021:         if op == "upgrade_print_statement":
0022:             return self._upgrade_print_statements(code)
0023:         if op == "upgrade_backtick_repr":
0024:             return self._upgrade_backtick_repr(code)
0025:         if op == "upgrade_apply_builtin":
0026:             return self._upgrade_apply_builtin(code)
0027:         if op == "upgrade_xrange":
0028:             return re.sub(r"\bxrange(?:\s*\()", "range", code)
0029:         if op == "upgrade_raw_input":
0030:             return re.sub(r"\braw_input(?:\s*\()", "input", code)
0031:         if op == "upgrade_except_syntax":
0032:             return re.sub(r"except\s+([^\n]+),\s*([A-Za-z_]\w*)\s*:", r"except \1 as \2:", code)
0033:         if op == "upgrade_exec_statement":
0034:             return self._upgrade_exec_statement(code)
0035:         if op == "upgrade_not_equal":
0036:             return code.replace("<>", "!=")
0037:         if op == "upgrade_iteritems":
0038:             return re.sub(r"\.iteritems(?:\s*\()", ".items", code)
0039:         if op == "upgrade_iterkeys":
```

```

0040:         return re.sub(r"\.iterkeys(?\s*\()", ".keys", code)
0041:     if op == "upgrade_intervalvalues":
0042:         return re.sub(r"\.intervalvalues(?\s*\()", ".values", code)
0043:     if op == "upgrade_has_key":
0044:         return re.sub(r"([A-Za-z_][\w\.]*)\.has_key\(((\[^\]]+)\))", r"\2 in \1", code)
0045:     if op == "upgrade_unicode":
0046:         return re.sub(r"\bunicode\b", "str", code)
0047:     if op == "upgrade_basestring":
0048:         return re.sub(r"\bbasestring\b", "str", code)
0049:     if op == "upgrade_long":
0050:         return re.sub(r"\blong\b", "int", code)
0051:     return code
0052:
0053: def _upgrade_print_statements(self, code: str) -> str:
0054:     upgraded_lines: List[str] = []
0055:     for line in code.splitlines():
0056:         stripped = line.lstrip()
0057:         indent = line[: len(line) - len(stripped)]
0058:
0059:         if not stripped.startswith("print ") or stripped.startswith("print ("):
0060:             upgraded_lines.append(line)
0061:             continue
0062:
0063:         if stripped.startswith("print >>"):
0064:             upgraded_lines.append(f"{indent}# TODO: manual migration required for redirected p
0065:             continue
0066:
0067:         payload = stripped[6:].rstrip()
0068:         if payload.endswith(","):
0069:             payload = payload[:-1].rstrip()
0070:         upgraded_lines.append(f"{indent}print({payload})")
0071:
0072:     return "\n".join(upgraded_lines) + ("\n" if code.endswith("\n") else "")
0073:
0074: def _upgrade_backtick_repr(self, code: str) -> str:
0075:     return re.sub(r"``([\^\\n]+)`", r"repr(\1)", code)
0076:
0077: def _upgrade_apply_builtin(self, code: str) -> str:
0078:     def replace(match):
0079:         fn_name = match.group(1).strip()
0080:         args_expr = match.group(2).strip()
0081:         kwargs_expr = match.group(3)
0082:         if kwargs_expr:
0083:             return f"{fn_name}(*{args_expr}, **{kwargs_expr.strip()})"
0084:         return f"{fn_name}(*{args_expr})"
0085:
0086:     pattern = re.compile(
0087:         r"\bbapply\s*\(\s*([A-Za-z_][\w\.]*)\s*,\s*(\[^\,][^\,]*\s*(?:,\s*([^\,]+)?)\s*)?\s*\)"
0088:     )
0089:     return pattern.sub(replace, code)
0090:
0091: def _upgrade_exec_statement(self, code: str) -> str:
0092:     upgraded_lines: List[str] = []
0093:     for line in code.splitlines():
0094:         stripped = line.lstrip()
0095:         indent = line[: len(line) - len(stripped)]
0096:
0097:         if not stripped.startswith("exec ") or stripped.startswith("exec("):
0098:             upgraded_lines.append(line)
0099:             continue
0100:
0101:         payload = stripped[5:].strip()
0102:         if " in " in payload:
0103:             expr, env = payload.split(" in ", 1)
0104:             parts = [part.strip() for part in env.split(",")]
0105:             if len(parts) == 2:
0106:                 upgraded_lines.append(f"{indent}exec({expr.strip()}, {parts[0]}, {parts[1]})")
0107:                 continue
0108:             if len(parts) == 1 and parts[0]:

```

```
0109:             upgraded_lines.append(f"{indent}exec({expr.strip()}, {parts[0]})")
0110:             continue
0111:
0112:         upgraded_lines.append(f"{indent}exec({payload})")
0113:
0114:     return "\n".join(upgraded_lines) + ("\n" if code.endswith("\n") else "")
```

Excerpt 3: Validation Core

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/validation.py

Simple explanation: this block handles validation gate and rollback decision logic. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `validate`, `_check_lint`, `_verify_behavior`, `_get_function_names`.
- Regex operations are present, so edge-case pattern handling should be tested carefully.
- AST usage is present, indicating structured syntax analysis/validation behavior.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Dict, Any, List
0004:
0005:
0006: LEGACY_REMAINDER_PATTERNS = {
0007:     "backtick_repr": re.compile(r"``[^\n]+`"),
0008:     "apply_builtin": re.compile(r"\bapply\s*\("),
0009:     "xrange": re.compile(r"\bxrange\s*\("),
0010:     "raw_input": re.compile(r"\braw_input\s*\("),
0011:     "iteritems": re.compile(r"\.iteritems\s*\("),
0012:     "iterkeys": re.compile(r"\.iterkeys\s*\("),
0013:     "itervalues": re.compile(r"\.itervalues\s*\("),
0014:     "has_key": re.compile(r"\.has_key\s*\("),
0015:     "not_equal": re.compile(r"<>"),
0016:     "except_comma": re.compile(r"except\s+([^\n:]+),\s*([A-Za-z_]\w*)\s*:"),
0017:     "exec_statement": re.compile(r"^\s*exec\s+[^\n].*$", re.MULTILINE),
0018: }
0019:
0020:
0021: class ValidationCore:
0022:     def validate(self, original_code: str, new_code: str) -> Dict[str, Any]:
0023:         warnings: List[str] = []
0024:
0025:         try:
0026:             ast.parse(new_code)
0027:         except SyntaxError as exc:
0028:             return {
0029:                 "success": False,
0030:                 "stage": "SYNTAX",
0031:                 "error": f"Syntax Error: {exc.msg} at line {exc.lineno}",
0032:                 "warnings": warnings,
0033:             }
0034:
0035:         lint_result = self._check_lint(new_code)
0036:         warnings.extend(lint_result.get("warnings", []))
0037:         if not lint_result["success"]:
0038:             return {
0039:                 "success": False,
```

```

0040:         "stage": "LINT",
0041:         "error": lint_result["error"],
0042:         "warnings": warnings,
0043:     }
0044:
0045:     behavior_result = self._verify_behavior(original_code, new_code)
0046:     warnings.extend(behavior_result.get("warnings", []))
0047:     if not behavior_result["success"]:
0048:         return {
0049:             "success": False,
0050:             "stage": "BEHAVIOR",
0051:             "error": behavior_result["error"],
0052:             "warnings": warnings,
0053:         }
0054:
0055:     return {
0056:         "success": True,
0057:         "stage": "COMPLETE",
0058:         "message": "Modernization validation passed.",
0059:         "warnings": warnings,
0060:     }
0061:
0062: def _check_lint(self, code: str) -> Dict[str, Any]:
0063:     warnings: List[str] = []
0064:     leftovers = [name for name, pattern in LEGACY_REMAINDER_PATTERNS.items() if pattern.search(
0065:     if leftovers:
0066:         return {
0067:             "success": False,
0068:             "error": f"Legacy constructs still remain after migration: {'', '.join(leftovers)}",
0069:             "warnings": warnings,
0070:         }
0071:
0072:     if re.search(r"(?<!/)/(?!/)", code):
0073:         warnings.append("Division operators remain; review Python 2 to 3 semantics manually.")
0074:
0075:     return {"success": True, "warnings": warnings}
0076:
0077: def _verify_behavior(self, original: str, new: str) -> Dict[str, Any]:
0078:     warnings: List[str] = []
0079:     original_functions = self._get_function_names(original)
0080:     new_functions = self._get_function_names(new)
0081:
0082:     missing = original_functions - new_functions
0083:     if missing:
0084:         return {
0085:             "success": False,
0086:             "error": f"Original functions disappeared during migration: {sorted(missing)}",
0087:             "warnings": warnings,
0088:         }
0089:
0090:     if "unicode" in original or "basestring" in original:
0091:         warnings.append("String type migration applied; bytes/text compatibility should be reviewed.")
0092:
0093:     return {"success": True, "warnings": warnings}
0094:
0095: def _get_function_names(self, code: str) -> set:
0096:     try:
0097:         tree = ast.parse(code)
0098:     except SyntaxError:
0099:         return set()
0100:     return {node.name for node in ast.walk(tree) if isinstance(node, ast.FunctionDef)}

```

Excerpt 4: Run Store + Reporting

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/run_store.py

Simple explanation: this block handles run persistence and retrieval contracts. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=6, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `__init__`, `save_run`, `list_runs`, `get_run`.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import json
0002: import os
0003: from datetime import datetime
0004: from typing import Dict, List, Optional
0005: from uuid import uuid4
0006:
0007:
0008: class RunStore:
0009:     def __init__(self, file_path: str):
0010:         self.file_path = file_path
0011:         os.makedirs(os.path.dirname(self.file_path), exist_ok=True)
0012:         if not os.path.exists(self.file_path):
0013:             with open(self.file_path, "w", encoding="utf-8"):
0014:                 pass
0015:
0016:     def save_run(self, payload: Dict) -> Dict:
0017:         record = {
0018:             "run_id": payload.get("run_id") or uuid4().hex,
0019:             "created_at": payload.get("created_at") or datetime.utcnow().isoformat() + "Z",
0020:             **payload,
0021:         }
0022:         with open(self.file_path, "a", encoding="utf-8") as handle:
0023:             handle.write(json.dumps(record, ensure_ascii=True) + "\n")
0024:         return record
0025:
0026:     def list_runs(self, limit: int = 20) -> List[Dict]:
0027:         runs = self._read_all()
0028:         runs.reverse()
0029:         return [self._summarize(run) for run in runs[:limit]]
0030:
0031:     def get_run(self, run_id: str) -> Optional[Dict]:
0032:         for run in reversed(self._read_all()):
0033:             if run.get("run_id") == run_id:
0034:                 return run
0035:         return None
0036:
0037:     def _read_all(self) -> List[Dict]:
0038:         runs = []
0039:         with open(self.file_path, "r", encoding="utf-8") as handle:
0040:             for line in handle:
0041:                 line = line.strip()
0042:                 if line:
```

```
0043:             runs.append(json.loads(line))
0044:         return runs
0045:
0046:     def _summarize(self, run: Dict) -> Dict:
0047:         return {
0048:             "run_id": run.get("run_id"),
0049:             "created_at": run.get("created_at"),
0050:             "probable_source_version": run.get("probable_source_version"),
0051:             "risk_score": run.get("risk_score"),
0052:             "validation_success": run.get("validation_success"),
0053:             "applied_transformations": run.get("applied_transformations", []),
0054:             "legacy_issue_count": run.get("legacy_issue_count", 0),
0055:         }
```